Time complexity of an algorithm can be computed in one of two ways

1. Step Count Method
2. Asymptotic Notation

Step Count Method

In this method, we count the no. of times each instruction is executed.

Based on that we will calculate the Time Complexity.

Example,

```
Algorithm Sum(A,n)
{
    S=0;                          ---- 1
    for(i=0; i<n; i++)            ---- n+1
    {                                  n = 2n+2   ] Actual total
        S=S+A[i];                 ---- n
    }
    return s;                     ---- 1
}
```

we focus on this

Assume n=5, A=8,9,7,2

i=0	
i=1	i was incremented 5 times & checked
i=2	
i=3	6 times
i=4	
i=5	

Total = 1+n+1+n+1

= 2n+3

Time complexity = O(n)

Space Complexity = S + n + A + i

1 word, 1 word, n word, 1 word

= n+3

O(n)

```
Linearsearch(arr, n, key)
{
    i = 0;                          ——— 1
    for(i = 0; i < n; i++)          ——— n+1
    {
        if(arr[i] == key)           ——— n
        {
            printf("Found");        ——— 1
        }
}
```

Total = 1 + n + 1 + n + 1

= 2n + 3

Time Complexity = O(n)

Space Complexity = i + n + key + arr

i → 1 word, n → 1 word, key → 1 word, arr → n word

= n + 3

O(n)

```
Algorithm Add(A,B,n)
{
    for(i=0; i<n; i++)              ——— n+1
    {
        for(j=0; j<n; j++)          ——— n(n+1)
        {
            C[i,j] = A[i,j] + B[i,j];   ——— n²
        }
    }
}
```

Total = n + 1 + n² + n + n²

= 2n² + 2n + 1

Time complexity = O(n²)

Space Complexity = i + n + A + B + C + i

i → 1 word, A → n² word

= 3n² + 3

O(n²)

```
Algorithm Multiply(A,B,n)
{
    for(i=0; i<n; i++)              ——— n+1
    {
        for(j=0; j<n; j++)          ——— n(n+1)
        {
            C[i,j] = 0;             ——— n²
            for(k=0; k<n; k++)      ——— n²(n+1)
            {
                C[i,j] = C[i,j] + A[i,k] * B[k,j];   ——— n³
            }
        }
    }
}
```

Time Complexity $= n+1+n^2+n+n^2+n^3+n^2+n^3$

$= 2n^3+3n^2+2n+1$

$= O(n^3)$

Space complexity $= A+B+C+i+j+k+n$

$n^2 \quad n^2 \quad n^2 \quad 1 \quad 1 \quad 1 \quad 1$

$= 3n^2+4$

$= O(n^2)$

```
for(i = 0; i < n; i++) {          ——— n+1
    for(j = 0; j < i; j++) {      ——— n(n-1)/2 + n
      operation;                  ——— n(n-1)/2
    }
}
```

value of i	How many times j runs? (condition check)	How many times operation run?
0	0 +1	0
1	1 +1	1
2	2 +1	2
⋮	⋮	⋮
n-1	n-1 +1	n-1
	$0+1+2+\cdots+n-1+n = \frac{n(n+1)}{2}$	$0+1+2+\cdots+n-1 = \frac{n(n-1)}{2}$

Total: $n+1+\frac{n^2+n}{2}+\frac{n^2-n}{2}$

$n^2+n+1 = O(n^2)$

```
p = 0;          ——— 1
for(i = 1; p <= n; i++) {
```

```
for(i = 1; p <= n; i++) {        ——— k+1
    p = p + i;                   ——— k
}
```

Iteration	value of i	value of p
1	1	0+1
2	2	0+1+2
3	3	0+1+2+3
⋮	⋮	⋮
k	k	$0+1+2+3+\cdots+k = \frac{k(k+1)}{2}$

Loop terminates when:

$$\frac{k(k+1)}{2} \leq n$$

$$k^2 \leq n$$

$$k \leq \sqrt{n}$$

$$\text{Total} = 1 + k + 1 + k = 2k + 2 = 2(\sqrt{n}) + 2$$

```
for(i = 1; i < n; i = i * 2) {   — log2(n)+1
    operation;                   — log2(n)
}
```

i	
1	2^0
2	2^1
4	2^2
⋮	⋮
2^k	2^k

Loop terminates when;

$$2^k \geq n$$

$$k \log_2(2) \leq \log_2(n)$$

$$k \leq \log_2(n)$$

$$\text{Total} = \log_2(n) + 1 + \log_2(n) = 2\log_2(n) + 1 \Rightarrow O(\log_2 n)$$

```
for(i = n; i >= 1; i = i / 2) {   — log2(n)+1
    operation;                    — log2(n)
}
```

i	
n	$n/2^0$
n/2	$n/2^1$
n/4	$n/2^2$
⋮	$n/2^k$

Loop terminates when;

$$\frac{n}{2^k} \geq 1$$

$$n \geq 2^k$$

$$k \leq \log_2(n)$$

```
for(i = 1; i* i < n; i++) {
    operation;
}
```

value of i

1
2
3
⋮
k

Loop terminates when

$k^2 < n$

$k < \sqrt{n}$

```
p=0 ——— 1
for(i = 1; i < n; i = i * 2) ——— log(n) +1
{
p++; ——— log(n)
}
for(j = 1; j < p; j = j * 2) ——— log(p) +1
{
   opertaion; ——— log(p)
}
```

value of i	value of p
1	1
2	2
4	3
⋮	⋮
2^k	k

loop terminates when;

$2^k < n$

$k \log_2(n) < \log_2(n)$

$k < \log_2(n)$

As $p = k$

so,

$p = \log_2(n)$

Value of j

1
2
⋮
2^k

$2^k < p$

$k < \log_2(p)$

$O(\log_2 p)$

or replacing

$O(\log_2(\log n))$

```
for(i = 0; i < n; i++) {
  for(j = 1; j < n; j = j * 2) {
  operation;
    }
}
```

n+1

$n(\log(n)+1)$

$n\log(n)$

Time complexity = $n+1+n\log n+n+n\log n$

$= 2n\log n + 2n + 1$

$O(n\log n)$